

4 Laboratorinis darbas. Slaptažodžių saugojimas

Kodėl slaptažodžiai neturėtų būti saugojami atviru tekstu?

- 1 Duomenų nutekėjimo atveju slaptažodžiai bus laisvai prieinami
- 2 Darbuotojai gali juos pasiekti ir panaudoti

Kodėl slaptažodžiai neturėtų būti šifruojami?

Užšifruotus slaptažodžius galima atšifruoti raktu. Jei užpuolikas ras raktą, šifravimas suteiks tokį pat saugumą kaip ir saugojimas atviru tekstu.

Kaip turėtų būti saugojami slaptažodžiai?

Aplikacijoms nereikia atvaizduoti slaptažodžio atviru tekstu (išskyrus labai retus atvejus). Vietoj slaptažodžio geriau saugoti įrodymą, kad vartotojas žino slaptažodį. Tai įgyvendinti galima maišos funkcijomis.

Maišos (angl. *hash*) funkcijos

Maišos funkcijas funkcijas žymesime h arba konkrečiu algoritmu, užšifruotą tekstą *hash*. Maišos funkcijos ima vieną parametą ir, dažniausiai, duoda fiksuoto ilgo rezultatą *hash*. Pagrindinės maišos funkcijų savybės:

- 1 Pirmvaizdžio atsparumas (angl. *Pre-image resistance*). Žinant *hash*, turėtų būti sunku rasti tokį atvirą tekstą m , kad $h(m) = \text{hash}$. t.y. funkcija yra vienkryptė.
- 2 Antrasis pirmvaizdžio atsparumas (angl. *Second pre-image resistance*). Žinant atvirą tekstą m_1 , turėtų būti sunku rasti kitą tokį tekstą m_2 , kad $h(m_1) = h(m_2)$.
- 3 Kolizijų atsparumas (angl. *Collision resistance*). Turėtų būti sunku rasti tokius atvirus tekstus m_1 ir m_2 , kad $h(m_1) = h(m_2)$.

Naujos paskyros sukūrimo procesas

- 1 Vartotojas į serverį nusiunčia vartotojo vardą *username* ir slaptažodį *password*
- 2 Serveris apskaičiuoja $hash = h(password)$
- 3 Duomenų bazėje išsaugoma *username* ir *hash*

Autentifikacijos procesas

- 1 Vartotojas į serverį nusiunčia vartotojo vardą *username* ir slaptažodį *password*
- 2 Serveris apskaičiuoja $hash = h(password)$
- 3 Gauta *hash* reikšmė palyginama su atitinkamo *username* *hash* reikšme duomenų bazėje

id	username	password	...
1	user1	7c6a180b36896a0a8c02787eeafb0e4c	...
2	user2	6cb75f652a9b52798eb6cf2201057c73	...
3	user3	819b0643d6b89dc9b579fd9c9094f28e	...
4	user4	7c6a180b36896a0a8c02787eeafb0e4c	...
...	...	...	...

Slaptažodžių *hash* saugumas

Saugant slaptažodžius, vien maišos funkcijų naudojimas neužtikrina slaptažodžių saugumo. Vartotojai daro daug klaidų kurdami slaptažodžius:

- 1 Naudoja populiarius slaptažodžius,
- 2 Naudoja tuos pačius slaptažodžius skirtingose sistemose,
- 3 Naudoja per trumpus slaptažodžius,
- 4 Nenaudoja slaptažodžių valdymo sistemų (angl. *password manager*).

Nutekėjus tokių slaptažodžių *hash* sąrašui, nesaugūs slaptažodžiai gali būti lengvai „nulaužti“. Nepaisant to, slaptažodžių *hash* saugoti naudinga. *Hash*'avimas visiškai apsaugo saugius slaptažodžius, taip pat leidžia mums išlošti laiko tinkamai sureaguoti, jei slaptažodžiai būtų nutekinti.

Dictionary atakos

Dictionary atakos - tai tokia *brute force* ataka, kai užpuolikas tikrina ar jo pasirinkto slaptažodžio *hash* atitinka kurį nors duomenų bazėje saugomo slaptažodžio *hash*.

Dictionary atakos procesas:

- 1 Pasirenkamas slaptažodis *password*, kurį manoma turės koks nors vartotojas
- 2 Apskaičiuojama $hash = h(password)$
- 3 Patikrinama ar gauta reikšmė lygi kuriai nors duomenų bazėje saugotai slaptažodžio *hash* reikšmei. Jei reikšmės lygios, teigiama, kad slaptažodis „nulaužtas“.
- 4 Grįžtama į 1 žingsnį ir pasirenkamas naujas slaptažodis.

Slaptažodžių žodynai

Slaptažodžių žodynai naudojami *dictionary* atakose sudaromi įvairiais būdais:

- 1 Surenkami anksčiau nutekinti slaptažodžiai
- 2 Naudojami populiarių slaptažodžių sąrašai, pvz. `rockyou.txt`
- 3 Išbandomi skirtingi variantai pagal iš anksto nustatytus šablonus.

Rainbow table atakos

Rainbow table atakos yra panašios į *dictionary* atakas, tik slaptažodžių *hash* apskaičiuojami iš anksto. Atakos procesas:

- 1 Užpuolikas sugeneruoja arba parsisiunčia failą su pasirinktomis simbolių sekomis ir jų *hash*
- 2 Patikrina ar nežinomo slaptažodžio *hash* yra įtrauktas į failą
- 3 Jei yra, teigiama, kad slaptažodis „nulaužtas“.

Jau sugeneruotus *rainbow tables* galima rasti <https://freerainbowtables.com/>.

Salting

Druska (angl. *salt*) yra kiekvienam vartotojui unikali, atsitiktinai sugeneruota simbolių eilutė, pridedama prie slaptažodžio *hash* proceso metu.

$$hash = h(password || salt)$$

Salt dažniausiai saugojamas kartu su slaptažodžiu.

Salt naudojimas sulėtina *dictionary* atakas, taip pat visiškai apsaugo nuo *rainbow table* atakų.

id	username	password	salt	...
1	user1	0bd402cf81cd3b4197598f340df6f4f3	gqLp1HnCKC	...
2	user2	ca0f18a81073fdfd56d05a0b49f9e871	0dgrfGFG0v	...
3	user3	908c8086bb5fd0e16b7799edefc84f96	Fnhik6IBA6	...
3	user4	5fdb60bebe521e02992a5e02c4b40f97	8wCAyBdjJR	...
...	...	...	...	...

Peppering

Pipiras (angl. *pepper*) yra slapta atsitiktinai sugeneruota simbolių eilutė, pridedama prie slaptažodžio *hash* proceso metu. Gali būti naudojama kartu su *salt*.

$$hash = h(password||salt||pepper)$$

Savybės:

- 1 *Pepper* visiems slaptažodžiams vienodas,
- 2 Kitaip nei *salt*, *pepper* neturėtų būti saugojamas duomenų bazėje
- 3 Turėtų būti saugojamas paslapčių saugykloje (angl. *secrets vault*)
- 4 Turėtų būti periodiškai atnaujinamas

Work factor

Darbo faktorius (angl. *work factor*) yra skaičius, nurodantis kiek iteracijų naudoja maišos funkcija (dažniausiai 2^{work}). *Work factor* dažniausiai saugomas kartu su *hash*. Jis sulėtina algoritmo darbą, kas taip pat sulėtina užpuoliką bandantį nulaužti slaptažodį.

Work factor parinkimas

Work factor dydis turėtų būti nustatomas pagal serverio pajėgumą ir vartotojų skaičių. Dydis turėtų būti parinktas toks, kad maišos funkcijos skaičiavimo procesas užimtų reikšmingą kiekį serverio resursų, bet nesulėtintų serverio darbo. Maišos funkcijos skaičiavimas neturėtų trukti ilgiau nei sekundę.

MD5

MD5 - maišos funkcija grąžinanti 128 bitų rezultatą

$\text{MD5}(\text{password}) = 7c6a180b36896a0a8c02787eeafb0e4c$

Dažnai naudojama senose sistemose. 2008 *CMU Software Engineering Institute* padarė išvadą, kad MD5 yra „kriptografiškai nulaužtas ir netinkamas naudojimui“. Egzistuoja kolizijų ataka, galinti rasti du skirtingus tekstus, grąžinančius tuos pačius *hash*.

SHA

SHA yra maišos funkcijų šeima sudaryta iš 4 šeimų. SHA-0 ir SHA-1 šeimose yra rasta kolizijų. Populiariausi algoritmai SHA-256 ir SHA-512 priklausantys SHA-2 šeimai.

$\text{SHA-256}(\text{password}) =$
 $0b14d501a594442a01c6859541bcb3e8164d183d32937b851835442f69d5c94e$

MD5 ir SHA trūkumai

Abu algoritmai yra per greitai slaptažodžių *h* skaičiavimui. Taip pat, negalima nurodyti *work factor*.

Argon2

Argon2 yra rakto išvedimo funkcija (angl. *key derivation function* arba *KDF*), 2015 laimėjęs *Password Hashing Competition*. Algoritmų šeimą sudaro 3 algoritmai:

- 1 *Argon2d*. Atsparus atakoms naudojančioms GPU, bet įveda galimas šoninio kanalo (angl. *side-channel*) atakas.
- 2 *Argon2i*. Atsparus *side-channel* atakoms.
- 3 *Argon2id*. Hibridinė ankstesnių algoritmų versija. Rekomenduojama naudoti, jei nežinote kuo algoritmai skiriasi arba manote, kad *side-channel* atakos yra problema.

Privalumai

- 1 Automatinis *salt*
- 2 *Work factor* nustatyti naudojami 3 parametrai: minimalus atminties kiekis (m), minimalus iteracijų skaičius (t), lygiagretinimo laipsnis (p).
- 3 Visi parametrai saugomi pačiame *hash*

Rekomenduojami *work factor* parametrai

- 1 $m=47104$ (46 MiB), $t=1$, $p=1$ (Nenaudoti su Argon2i)
- 2 $m=19456$ (19 MiB), $t=2$, $p=1$ (Nenaudoti su Argon2i)
- 3 $m=12288$ (12 MiB), $t=3$, $p=1$
- 4 $m=9216$ (9 MiB), $t=4$, $p=1$
- 5 $m=7168$ (7 MiB), $t=5$, $p=1$

Visos konfigūracijos turėtų duoti panašų saugumą.

Struktūra

Argon2id(password) =

\$argon2id\$v=19\$m=16,t=2,p=1\$TGg50Es4VThzN1JaQkk40A\$PFicMHYBQhge/SASZ9rwBw

- argon2id - algoritmas
- v=19 - versijos numeris
- m=16,t=2,p=1 - *work factor* parametrai
- TGg50Es4VThzN1JaQkk40A - *salt*
- PFicMHYBQhge/SASZ9rwBw - slaptažodžio *hash*

scrypt

Turėtų būti naudojamas tokiose sistemose, kuriose nėra galimybės naudoti *Argon2*.

Privalumai

- 1 Turi 3 *work factor* parametrus: minimalus CPU/RAM resursai (N), bloko dydis (r) lygiagretinimo laipsnis (p).

Rekomenduojami *work factor* parametrai

- 1 $N = 2^{17}$ (128 MiB), $r = 8$ (1024 bytes), $p = 1$
- 2 $N = 2^{16}$ (64 MiB), $r = 8$ (1024 bytes), $p = 2$
- 3 $N = 2^{15}$ (32 MiB), $r = 8$ (1024 bytes), $p = 3$
- 4 $N = 2^{14}$ (16 MiB), $r = 8$ (1024 bytes), $p = 5$

Trūkumai

- 1 *Work factor* parametrai nėra saugomi kartu su *hash*
- 2 Nėra automatinio *salt*

bcrypt

Naudojamas *legacy* sistemose, kuriose nėra galimybės naudoti *argon2* arba *scrypt*.

Privalumai

- 1 Automatinis *salt*
- 2 Visi parametrai saugomi kartu su *hash*

Trūkumai

- 1 Maksimalus slaptažodžio ilgis 72 baitai. Apribojimą galima apeiti naudojant *prehash*.
- 2 *Work factor* galima nustatyti tik vienu parametru. Rekomenduojama reikšmė bent 10.

Struktūra

```
bcrypt(password) =  
$2a$12$R9h/cIPz0gi.URNNX3kh20PST9/PgBkqquzi.Ss7KIUg02t0jWMUW
```

- 2a - versijos numeris
- 12 - *work factor* 2^{12}
- R9h/cIPz0gi.URNNX3kh20 - *salt*
- PST9/PgBkqquzi.Ss7KIUg02t0jWMUW - slaptažodžio *hash*

Maišos funkcijų implementacija

Kad ir kokia maišos funkcija bus pasirinkta slaptažodžių *hash* skaičiavimui, ateityje ją reikės keisti. Perėjimo metu turėtų būti galima naudoti ir seną ir naują maišos funkcijas. Naudoti kelis algoritmus vienu metu yra lengviau, jei slaptažodžio *hash*, algoritmo tipas ir *work factor* yra saugomi kartu.

Simbolių palaikymas

Slaptažodžių *hash* skaičiavimui naudojamos maišos funkcijos turėtų palaikyti šiuos simbolius:

- 1 *Unicode* simbolius
- 2 Įvairių kalbų simbolius
- 3 Piktogramas
- 4 NULL

Pirma strategija

Procesas:

- 1 Nustatyti nebegaliojančiais (ir ištrinti) visus slaptažodžius.
- 2 Vartotojui bandant prisijungti, nurodyti pasikeisti slaptažodį ir išsaugoti naują *hash*.

Trūkumai:

- 1 Klientų aptarnavimo skyrius gali sulaukti daug skambučių
- 2 Klientai gali pagalvoti, kad buvo nutekinti jų duomenys

Antra strategija

Panaudoti dabar saugomus *hash*, kaip parametrus saugesnėms maišos funkcijoms. Pvz. dabar duomenų bazėje saugoma `MD5(password)`, šiuos *hash* galima pakeisti `bcrypt(MD5(password))`. Taip išvengiama poreikio žinoti slaptažodį atviru tekstu, bet slaptažodžius bus galimai lengviau nulaužti. Vartotojui kitą kartą prisijungus, *hash* turėtų būti perskaičiuotas.